Universidad Tecnológica de Campeche

Ingeniería en Desarrollo y Gestión de Software

Práctica en equipo · Actividad 2

Plan, casos y ejecución de pruebas

Caso de estudio: Sistema de control de inventario

Equipo

Emily Yamilett Pinzón Sánchez · Gestora de proyectos

Chuina Guadalupe García Campos · Analista y Tester

Pablo Alejandro Magaña Hernández · Desarrollador / DevOps

Emmanuel Hernández Barbis · Diseñador

Ana Edith Rodríguez Pérez · Cliente (aceptación)

San Francisco de Campeche, Campeche · 28 de octubre de 2026

Práctica en equipo.

Caso de estudio: Sistema de control de inventario

Un equipo desarrolla un sistema para registrar productos, consultar existencias y actualizar cantidades después de cada venta.

Tomamos como base el sistema de inventario que el equipo ya tiene (PHP 8, API JSON y archivo de datos JSON) y sobre él escribimos los requisitos, el plan, los casos y el flujo de integración continua que pide la práctica. Todas las pruebas están automatizadas con PHPUnit y se encuentran en la carpeta sistema/tests de esta entrega.

- **Código y pruebas:** Actividad 2/sistema
- **Evidencia de la ejecución:** sistema/evidencias (JUnit, reporte HTML, JSON y salida de consola)
- **Flujo de CI:** sistema/.github/workflows/ci.yml
- **Documento y diagramas editables:** sistema/docs (Word, PDF y PowerPoint)
- **Repositorio en GitHub:** https://github.com/azharsonix/sistema-inventario-actividad-2

1. Identificar cinco requisitos funcionales del sistema.

Identificamos cinco requisitos funcionales. Cada uno tiene un criterio de aceptación que se puede comprobar con una prueba.

ID	Requisito	Criterio de aceptación
RF-01	Registrar productos	El sistema guarda código, nombre, categoría, precio, existencia inicial y stock mínimo. El código es único (3 a 20 caracteres: A-Z, 0-9 o guion) y se guarda en mayúsculas. Rechaza códigos duplicados, precios negativos y nombres vacíos.
RF-02	Consultar existencias	Permite consultar un producto por su código sin importar mayúsculas o espacios y consultar la existencia de todos los productos. Si el código no existe muestra un mensaje claro.
RF-03	Actualizar la existencia con cada venta	Cada venta registra una salida que descuenta la cantidad vendida y guarda el movimiento con la existencia resultante. Rechaza cantidades de cero, negativas o mayores a la existencia sin modificar ningún dato.
RF-04	Registrar entradas de mercancía	Cada compra a proveedor suma la cantidad recibida a la existencia y queda en el historial de movimientos.
RF-05	Reporte de inventario y resurtido	El reporte muestra total de productos, total de unidades, valor del inventario y la lista de productos cuya existencia es igual o menor a su stock mínimo.

2. Elaborar un plan de pruebas con objetivo, alcance, recursos y responsables.

2.1 Objetivo

Comprobar que el sistema registra productos, consulta existencias y actualiza las cantidades con cada venta y cada entrada según RF-01 a RF-05; que rechaza los datos inválidos sin alterar el inventario, y que cada cambio que entra al repositorio conserva lo que ya funcionaba.

2.2 Alcance

Incluye	Clases Producto, Inventario, ReporteInventario y RepositorioInventario. API HTTP: /api/productos, /api/productos/{código}/salidas, /api/productos/{código}/entradas, /api/movimientos y /api/reporte. Guardado y lectura del archivo de datos JSON. Pipeline de integración continua en GitHub Actions.
No incluye	Interfaz gráfica (la probamos con Playwright en la entrega anterior). Usuarios y permisos, pruebas de carga y rendimiento, conexión con PostgreSQL.
Niveles	Unitarias, de integración, de sistema y de aceptación (sección 4).

2.3 Recursos

Recurso	Cuál
Humanos	Los cinco integrantes del equipo (sección 2.4).
Herramientas	PHP 8.4.27 (local) y PHP 8.2, 8.3 y 8.4 en CI · PHPUnit 11.5.56 · Composer 2.10 · GitHub Actions · VS Code · Trello para el seguimiento de defectos.
Ambiente local	Windows 11 Pro, servidor integrado de PHP (php -S) en un puerto libre que elige la prueba.
Ambiente de CI	Máquinas ubuntu-latest de GitHub Actions; una por cada nivel de prueba.
Datos de prueba	Cada prueba crea su propio archivo JSON temporal con productos conocidos (LAP-001, MOU-010, TEC-005, ACC-002, RED-001) y lo borra cuando acaba, de modo que las pruebas no dependen unas de otras ni de data/inventario.json.
Scripts	scripts/ejecutar-pruebas.php (ejecuta las cuatro suites y genera la evidencia) y scripts/simular-defectos.php (regresión con defectos inyectados).

2.4 Responsables

Integrante	Rol	Responsabilidad en las pruebas
Emily Yamilett Pinzón Sánchez	Gestora de proyectos	Aprueba el plan, revisa los Pull Requests y da seguimiento a los defectos en Trello.
Chuina Guadalupe García Campos	Analista y Tester	Diseña los casos, ejecuta las rondas de pruebas y registra resultados y defectos.
Pablo Alejandro Magaña Hernández	Desarrollador / DevOps	Automatiza las pruebas, mantiene el pipeline de CI y corrige los defectos.
Emmanuel Hernández Barbis	Diseñador	Revisa que los mensajes de error que recibe el usuario sean claros (CP-02, CP-04 y CP-08).
Ana Edith Rodríguez Pérez	Cliente	Valida los escenarios de aceptación CP-09 y CP-10 y firma su aprobación.

2.5 Criterios

Entrada	El código está en la rama, composer install termina sin errores y la rama base tiene el CI en verde.
Aprobación de un caso	El resultado obtenido es igual al esperado. Si aparece un error no controlado o una existencia cambia cuando no debía, el caso queda No aprobado.
Salida	10 de 10 casos aprobados, ningún defecto de prioridad Alta abierto y el pipeline de CI en verde.
Suspensión	Si el servidor de pruebas no levanta o falla la instalación de dependencias, se detiene la ronda hasta resolverlo.

3. Diseñar al menos seis casos de prueba.

Diseñamos diez casos. Cada uno está automatizado y lleva su ID en el nombre de la prueba (atributo TestDox), así el reporte y el CI muestran el mismo ID que este documento.

ID	RF	Precondición	Entrada / pasos	Resultado esperado
CP-01	RF-01	Inventario vacío	Registrar « mou-010 », Mouse inalámbrico, Periféricos, $250, 30 piezas, mínimo 5	Se registra con código MOU-010 y existencia 30; el inventario tiene 1 producto.
CP-02	RF-01	Existe LAP-001	Registrar «lap-001», Otra laptop	Error «Ya existe un producto con el código LAP-001.»; sigue habiendo 1 producto y conserva su nombre.
CP-03	RF-03	LAP-001 con existencia 10	Venta de 4 piezas	Existencia 6. Movimiento de salida: cantidad 4, existencia resultante 6, motivo «Venta».
CP-04	RF-03	LAP-001 con existencia 10	Venta de 50 piezas	Error «Existencia insuficiente de LAP-001…»; la existencia sigue en 10 y no se registra movimiento.
CP-05	RF-02, RF-03	Archivo JSON con LAP-001 (10) y MOU-010 (30)	Venta de 7 MOU-010 por el repositorio; abrir el archivo con un repositorio nuevo	Existencias LAP-001: 10 y MOU-010: 23; un movimiento con existencia resultante 23.
CP-06	RF-03	El mismo archivo JSON	Venta de 11 LAP-001 dentro de una actualización del repositorio	La venta se rechaza, el archivo queda idéntico y LAP-001 sigue en 10.
CP-07	RF-01, RF-02, RF-03	Servidor en ejecución con LAP-001	POST /api/productos (TEC-005, 14 piezas) → GET /api/productos/tec-005 → POST /salidas con cantidad 3 → GET	Alta 201; consulta con existencia 14; venta 201 con existencia 11 y motivo «Venta»; consulta final con 11.
CP-08	RF-03	Servidor en ejecución con LAP-001 (10)	POST /api/productos/LAP-001/salidas con cantidad 0, −2 y 11	Las tres responden 422 con mensaje de error; LAP-001 sigue en 10 y el historial está vacío.
CP-09	RF-05	ACC-002 (12, mín. 5), MOU-010 (30, mín. 10), RED-001 (6, mín. 4)	Ventas del día: 7 ACC-002, 2 RED-001 y 5 MOU-010; abrir /api/reporte	Por resurtir: ACC-002 y RED-001 (no MOU-010). 3 productos, 34 unidades, valor $10,291.00.
CP-10	RF-02, RF-04	Los mismos tres productos	Entrada de 10 MOU-010; venta de 3 en mostrador; consultar existencias e historial	MOU-010: 37 (ACC-002: 12, RED-001: 6). Historial: salida de 3 → 37 y entrada de 10 → 40.

3.1 Matriz de trazabilidad

Cada requisito queda cubierto por al menos un caso. RF-01, RF-02 y RF-03 tienen casos de dos niveles o más; RF-04 y RF-05 se cubren con un caso de aceptación cada uno.

RF	CP 01	CP 02	CP 03	CP 04	CP 05	CP 06	CP 07	CP 08	CP 09	CP 10
RF-01	✓	✓					✓			
RF-02					✓		✓			✓
RF-03			✓	✓	✓	✓	✓	✓		
RF-04										✓
RF-05									✓	

4. Clasificar los casos como pruebas unitarias, de integración, de sistema o de aceptación.

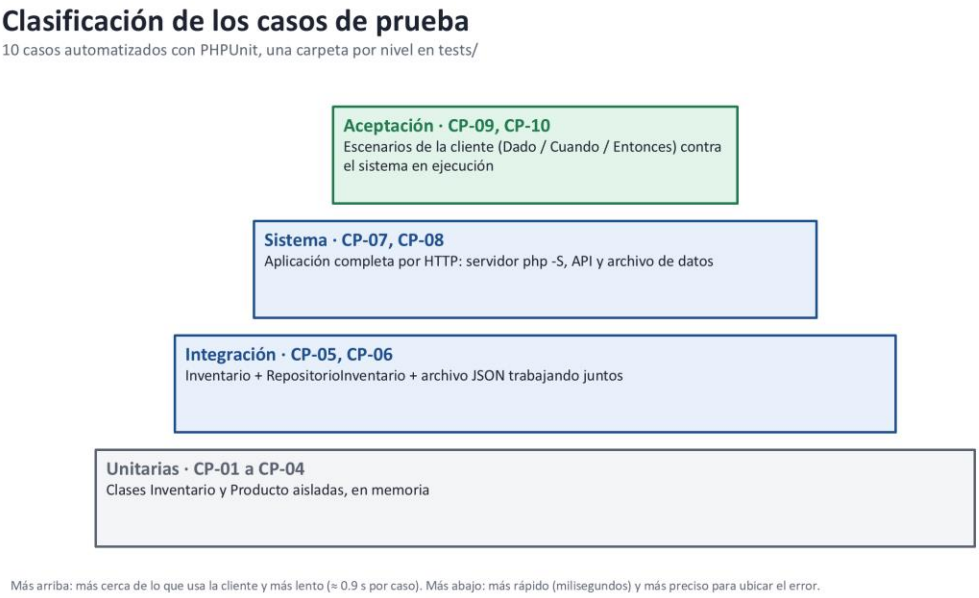


Figura 1. Niveles de prueba y casos de cada uno (diagrama editable en Diagramas - Actividad 2.pptx, diapositiva 3).

Tipo	Casos	Qué comprueba	Cómo se ejecuta
Unitaria	CP-01, CP-02, CP-03, CP-04	Una clase aislada (Inventario y Producto) da el resultado correcto.	En memoria, sin archivo ni servidor. tests/Unitarias. Tarda milisegundos.
Integración	CP-05, CP-06	Inventario, RepositorioInventario y el archivo JSON funcionan juntos: la venta se guarda y una venta rechazada no escribe nada.	Archivo JSON temporal con bloqueo (flock). tests/Integracion.
Sistema	CP-07, CP-08	La aplicación completa responde por HTTP: rutas, validación, códigos 201 y 422 y datos guardados.	La prueba levanta php -S con public/index.php en un puerto libre y le envía peticiones reales. tests/Sistema.
Aceptación	CP-09, CP-10	Escenarios de la cliente (Dado / Cuando / Entonces): cierre del día con resurtido y llegada de mercancía.	Contra el sistema en ejecución. tests/Aceptacion. Ana Edith revisa el resultado y lo aprueba.

5. Ejecutar las pruebas y registrar sus resultados.

Ejecutamos las cuatro suites con el script scripts/ejecutar-pruebas.php el 28 de octubre de 2026 (Windows NT 10.0, PHP 8.4.27, PHPUnit 11.5.56). El script corre cada nivel por separado, guarda el JUnit de cada uno y arma el reporte con el resultado, el requisito y el tiempo de cada caso.

```
cd "Actividad 2\sistema"
php scripts\ejecutar-pruebas.php --ronda="Ronda 1 - ejecución inicial"
```

ID	Tipo	Resultado obtenido	Estado	Tiempo
CP-01	Unitaria	Producto MOU-010 registrado con existencia 30; inventario con 1 producto.	Aprobada	0.008 s
CP-02	Unitaria	«Ya existe un producto con el código LAP-001.»; 1 producto, nombre Laptop Lenovo.	Aprobada	0.001 s
CP-03	Unitaria	Existencia 6; movimiento salida, 4 piezas, resultante 6, motivo Venta.	Aprobada	0.001 s
CP-04	Unitaria	«Existencia insuficiente de LAP-001: hay 10 y se pidieron 50.»; existencia 10; sin movimientos.	Aprobada	0.001 s
CP-05	Integración	Al recargar: LAP-001 10, MOU-010 23; 1 movimiento con resultante 23.	Aprobada	0.009 s
CP-06	Integración	Venta rechazada; el archivo no cambió; LAP-001 en 10.	Aprobada	0.002 s
CP-07	Sistema	201 → existencia 14 → 201 con existencia 11 y motivo Venta → existencia 11.	Aprobada	0.889 s
CP-08	Sistema	422, 422 y 422 con mensaje; LAP-001 en 10; historial vacío.	Aprobada	0.865 s
CP-09	Aceptación	Por resurtir: ACC-002 y RED-001; 3 productos, 34 unidades, $10,291.00.	Aprobada	0.880 s
CP-10	Aceptación	MOU-010 37, ACC-002 12, RED-001 6; historial: salida 3 → 37, entrada 10 → 40.	Aprobada	0.872 s

Resumen de la ronda 1: 10 casos ejecutados, 10 aprobados, 0 no aprobados; 43 aserciones en 3.528 s. Los casos de sistema y aceptación tardan cerca de un segundo porque cada uno levanta su propio servidor.

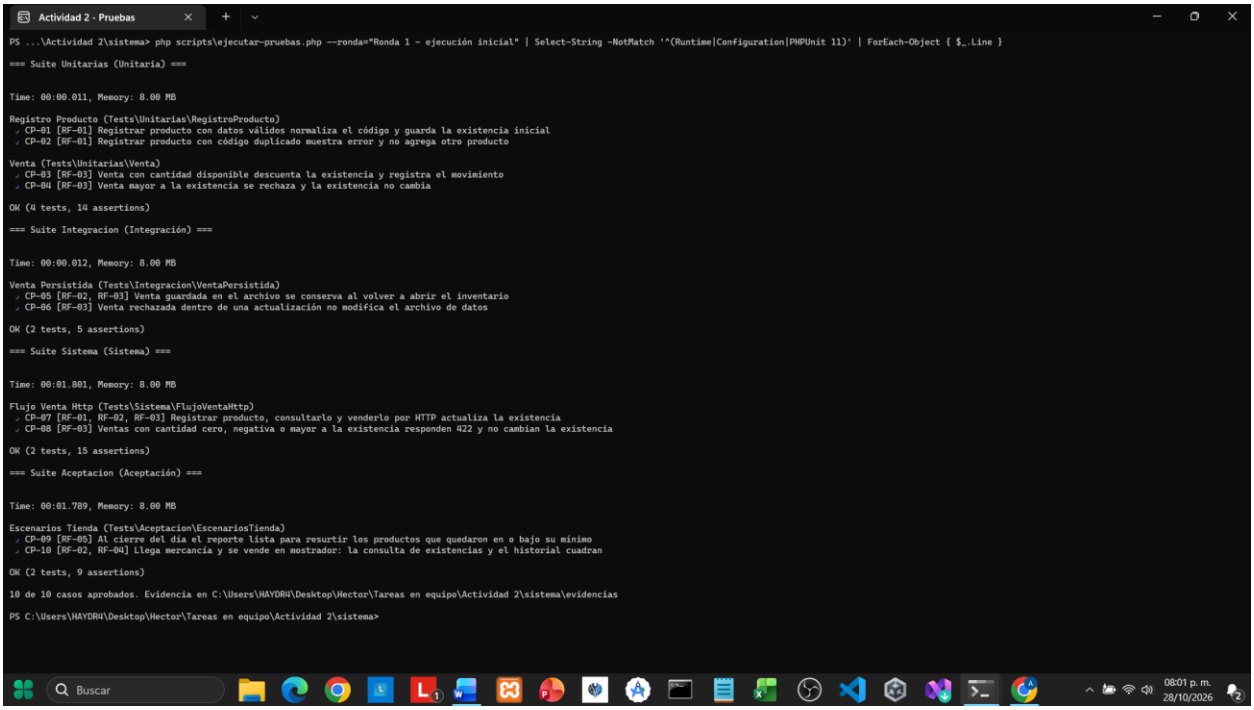


Figura 2. Captura de pantalla de la ejecución de la ronda 1 en Windows Terminal.

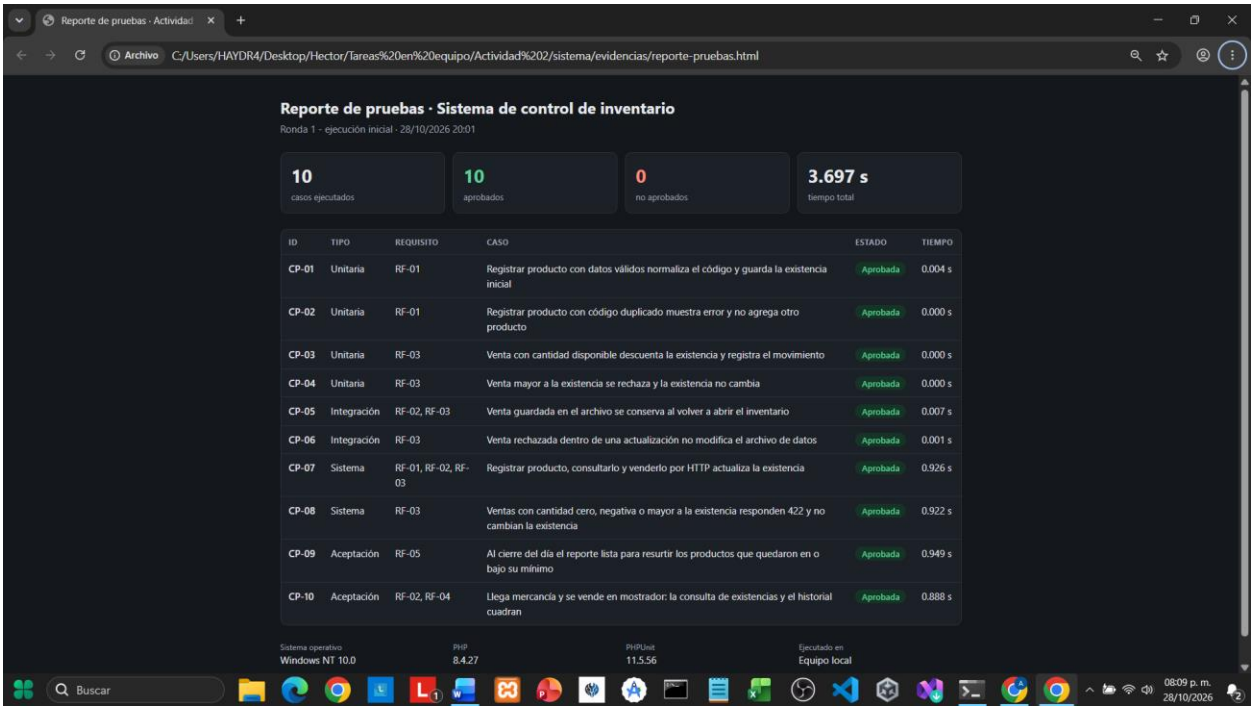


Figura 3. Captura de pantalla del reporte generado (evidencias/reporte-pruebas.html) abierto en Chrome.

Evidencia que deja cada ejecución en sistema/evidencias:

- reporte-pruebas.html: reporte con totales, estado y tiempo de cada caso.
- junit-Unitarias.xml, junit-Integracion.xml, junit-Sistema.xml y junit-Aceptacion.xml: formato que leen GitHub Actions y otras herramientas.
- resultados.json y resultados.md: los mismos resultados en datos y en tabla.
- salida-<Suite>.txt: salida de la consola de cada nivel.

6. Documentar dos defectos hipotéticos con descripción, prioridad y evidencia esperada.

Documentamos dos defectos hipotéticos que pueden aparecer en el sistema. Para que la evidencia esperada no se quede en suposición, el script scripts/simular-defectos.php los provoca en una copia de src/ (el código original no se toca) y ejecuta los diez casos contra esa copia. Los mensajes de la fila «Evidencia obtenida en la simulación» salen de esa ejecución.

DEF-01 · La venta deja la existencia en negativo

ID	DEF-01
Descripción	Al registrar una venta con una cantidad mayor a la existencia, el sistema la acepta, responde 201 y la existencia queda en negativo. La causa sería que Inventario::registrarSalida deja de comparar la cantidad con la existencia.
Requisito afectado	RF-03
Pasos para reproducir	1. LAP-001 con existencia 10. 2. POST /api/productos/LAP-001/salidas con {"cantidad": 11}. 3. GET /api/productos/LAP-001.
Resultado esperado	Respuesta 422 «Existencia insuficiente de LAP-001…»; existencia 10.
Resultado con el defecto	Respuesta 201; existencia −1 y un movimiento de salida con existencia resultante −1.
Severidad / prioridad	Crítica / Alta. El inventario deja de reflejar lo que hay en la tienda y se venden productos que no existen.
Casos que lo detectan	CP-04 (unitaria), CP-06 (integración) y CP-08 (sistema).
Evidencia esperada	CP-04, CP-06 y CP-08 en No aprobada dentro de evidencias/regresion/DEF-01/junit.xml. Job «Unitarias» en rojo en GitHub Actions (los niveles siguientes no se ejecutan) y el PR bloqueado. Respuesta JSON de GET /api/productos/LAP-001 con "existencia": -1.
Evidencia obtenida en la simulación	CP-04: Se esperaba InventarioException por existencia insuficiente. CP-06: Se esperaba que la venta se rechazara. CP-08: La venta de 11 unidades de LAP-001 (existencia 10) debía responder 422.
Responsable / estado	Pablo Magaña (corrección), Chuina García (verificación) / Cerrado tras la ronda de regresión.

DEF-02 · El reporte no marca para resurtir un producto que está justo en su mínimo

ID	DEF-02
Descripción	El reporte solo lista para resurtir los productos con existencia menor a su stock mínimo y deja fuera los que quedan exactamente en el mínimo. La causa sería que Producto::bajoStockMinimo usa < en lugar de <=.
Requisito afectado	RF-05
Pasos para reproducir	1. ACC-002 con existencia 12 y mínimo 5. 2. Vender 7 piezas (queda en 5). 3. GET /api/reporte.
Resultado esperado	ACC-002 aparece en «bajoStockMinimo» (productos a resurtir).
Resultado con el defecto	La lista «bajoStockMinimo» llega vacía y la tienda no pide mercancía a tiempo.
Severidad / prioridad	Moderada / Media. No altera existencias, pero puede provocar faltantes.
Casos que lo detectan	CP-09 (aceptación).
Evidencia esperada	CP-09 en No aprobada en evidencias/regresion/DEF-02/junit.xml con el mensaje de productos a resurtir. Respuesta JSON de /api/reporte con "bajoStockMinimo": []. Job «Aceptación» en rojo en GitHub Actions.
Evidencia obtenida en la simulación	CP-09: Productos a resurtir: se esperaban ACC-002 (5 de mínimo 5) y RED-001 (4 de mínimo 4).
Responsable / estado	Pablo Magaña (corrección), Ana Rodríguez (revalida CP-09) / Cerrado tras la ronda de regresión.

7. Proponer cómo se repetirán las pruebas después de corregir los defectos.

Proponemos este ciclo para cada defecto. Los pasos 4, 5 y 7 los ejecuta GitHub Actions de forma automática.

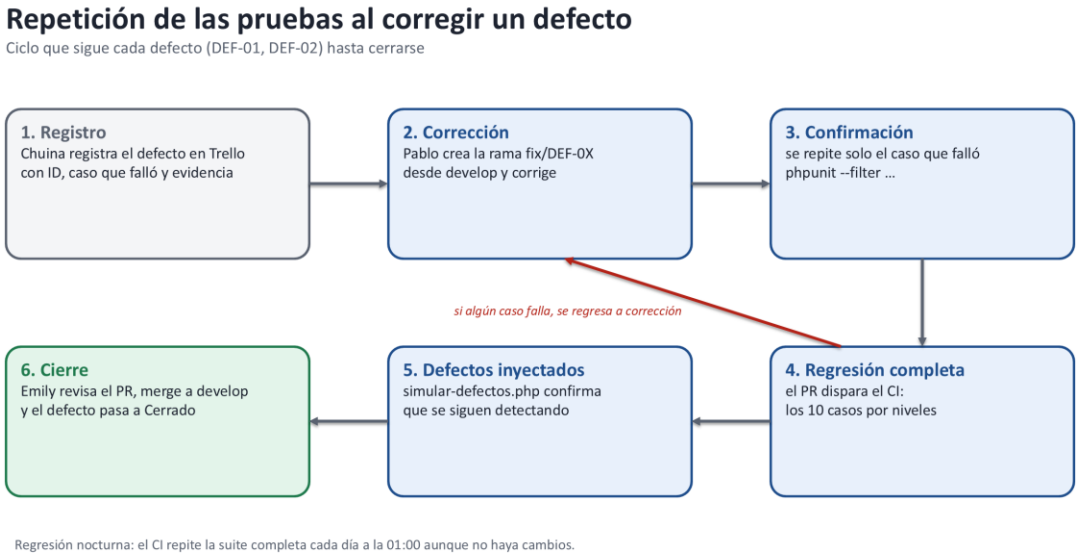


Figura 4. Ciclo para repetir las pruebas al corregir un defecto (diagrama editable, diapositiva 2).

1. **Registro.** Chuina registra el defecto en Trello con su ID, el caso que falló, la evidencia (JUnit o captura) y la prioridad.
2. **Corrección.** Pablo crea la rama fix/DEF-01 desde develop y corrige el código.
3. **Prueba de confirmación.** Se vuelve a ejecutar solo el caso que falló para comprobar la corrección: php vendor/bin/phpunit --filter testVentaMayorALaExistencia (CP-04).
4. **Regresión completa.** Al subir la rama y abrir el Pull Request, GitHub Actions ejecuta los diez casos por niveles. La suite completa tarda menos de 5 segundos, así que siempre se repite completa y no solo los casos relacionados.
5. **Regresión con defectos inyectados.** El job «Regresión DEF» vuelve a provocar DEF-01 y DEF-02 en una copia del código y exige que las pruebas los detecten. Si alguien borra o debilita CP-04 o CP-09, este job falla aunque el resto esté en verde.
6. **Cierre.** Con todos los checks en verde, Emily revisa el PR, se integra a develop y el defecto pasa a Cerrado en Trello. Para DEF-02, Ana revisa otra vez el reporte (CP-09).
7. **Regresión nocturna.** El CI repite la suite completa cada día a la 01:00 aunque no haya cambios.

7.1 Resultado de la ronda de regresión

Ejecutamos scripts/simular-defectos.php (composer regresion):

Ronda	Prioridad	Casos que deben fallar	Casos que fallaron	Resultado
DEF-01	Alta	CP-04, CP-06, CP-08	CP-04, CP-06, CP-08	Aprobada (detectado)
DEF-02	Media	CP-09	CP-09	Aprobada (detectado)
Código corregido	—	Los 10 casos	10 de 10 aprobados	Aprobada

Con el defecto presente fallan exactamente los casos que cubren ese requisito, y con el código corregido los diez casos vuelven a pasar.

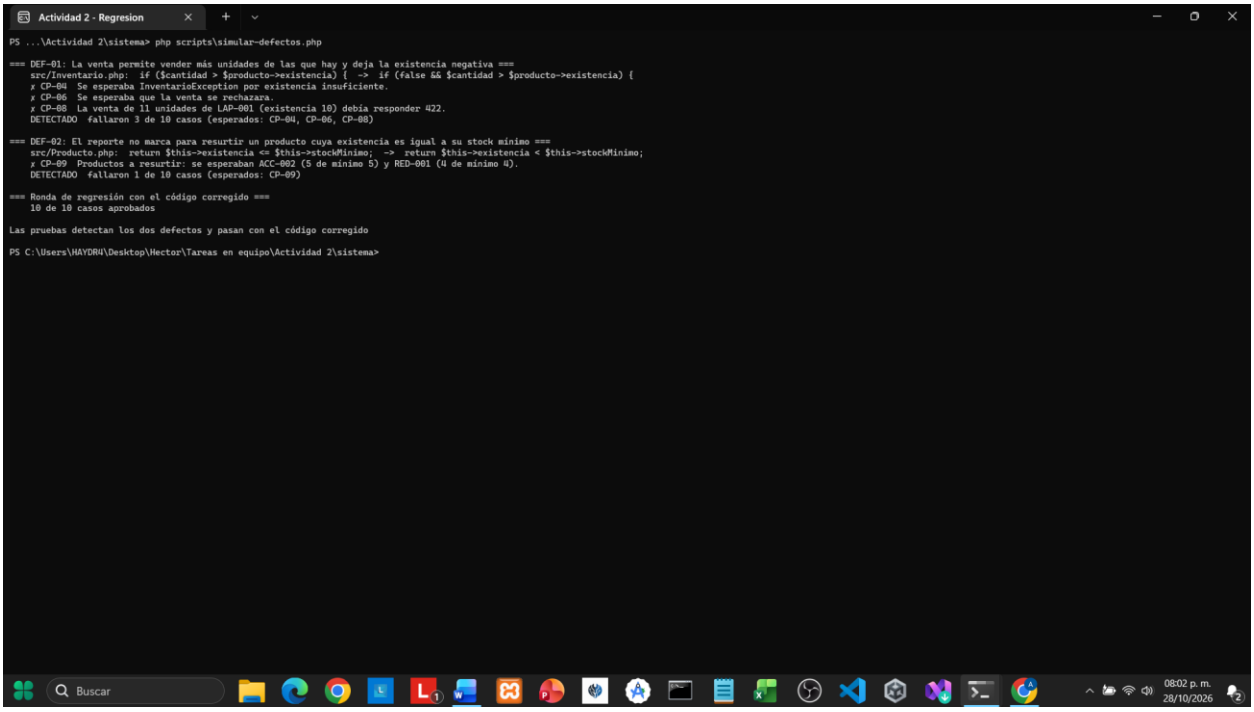


Figura 5. Captura de pantalla de la ronda de regresión con DEF-01 y DEF-02 inyectados.

8. Diseñar un flujo de integración continua para ejecutar automáticamente las pruebas cuando se actualice el repositorio.

El flujo está en sistema/.github/workflows/ci.yml y usa GitHub Actions. Se ejecuta en cada push a main, develop, feature/**, release/** y hotfix/**, en cada Pull Request hacia main o develop, cada noche y de forma manual. Los niveles van en cadena: si uno falla, los siguientes no corren y el PR no se puede integrar.

El sistema está publicado en GitHub y el flujo ya se ejecutó con todos los jobs en verde:

- **Repositorio:** https://github.com/azharsonix/sistema-inventario-actividad-2
- **Ejecución del CI:** https://github.com/azharsonix/sistema-inventario-actividad-2/actions/runs/40804500676 (8 de 8 jobs exitosos; la evidencia se descarga como artefacto «evidencia-pruebas»)
- **Historial de ejecuciones:** https://github.com/azharsonix/sistema-inventario-actividad-2/actions

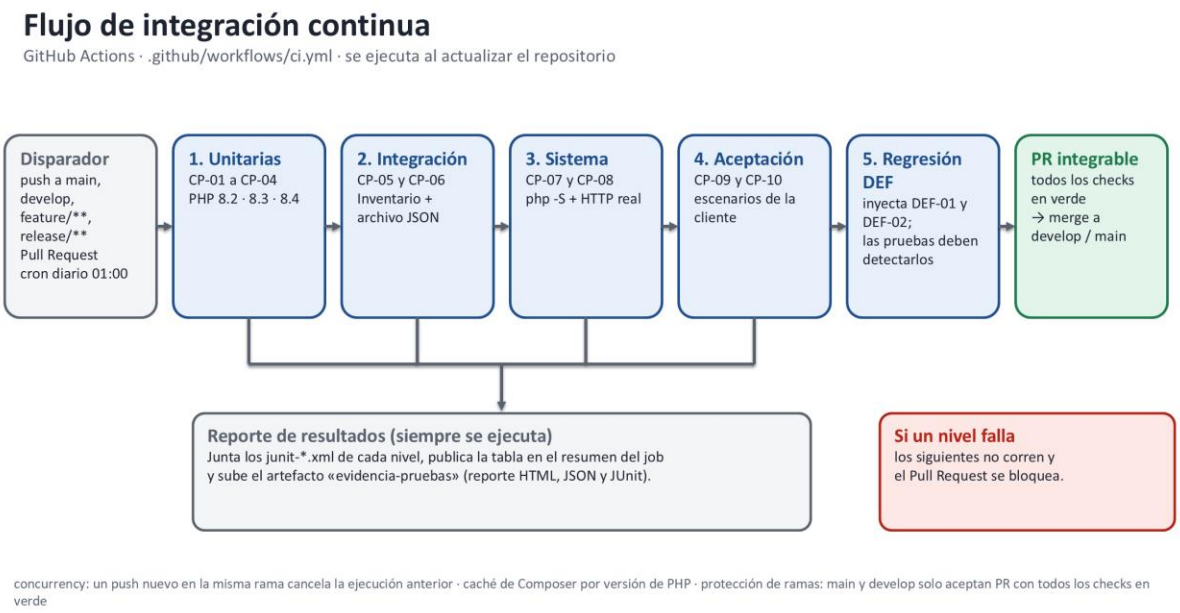


Figura 6. Jobs del pipeline y orden de ejecución (diagrama editable, diapositiva 1).

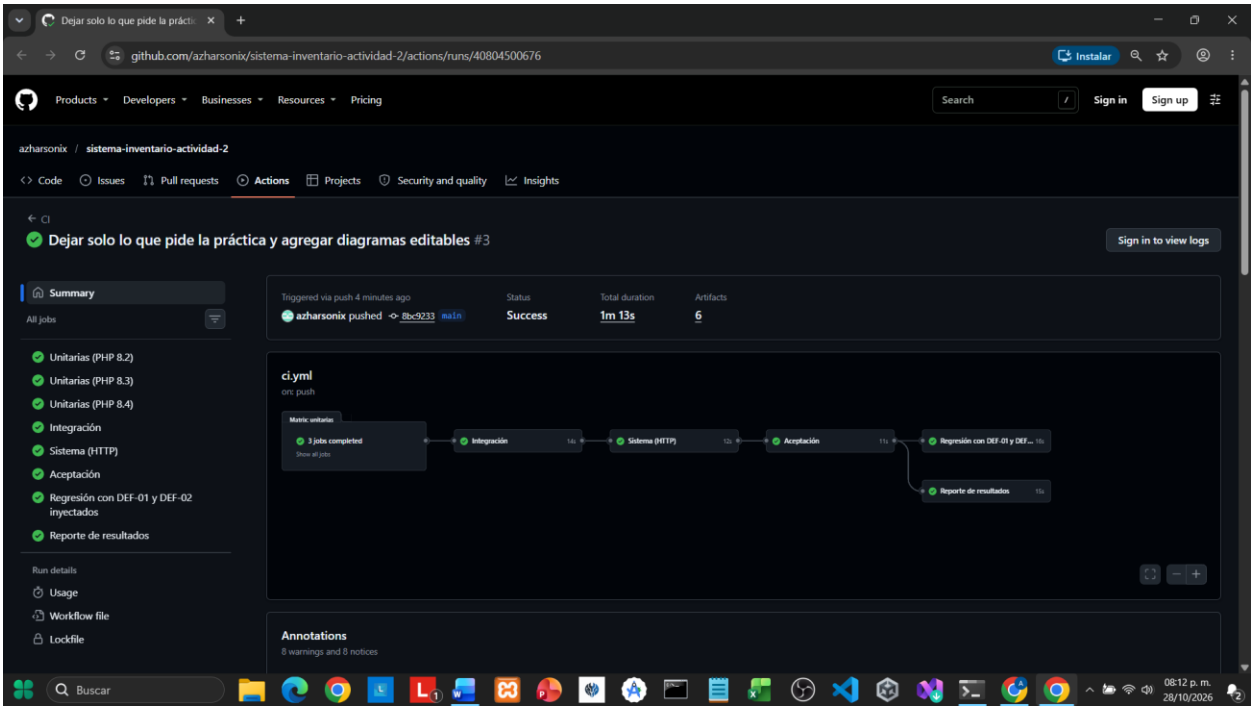


Figura 7. Captura de pantalla de la ejecución del CI en GitHub Actions.

Job	Depende de	Qué hace
Unitarias (PHP 8.2, 8.3 y 8.4)	—	Ejecuta CP-01 a CP-04 en las tres versiones de PHP (matriz) para detectar diferencias entre versiones.
Integración	Unitarias	Ejecuta CP-05 y CP-06 con archivo JSON real.
Sistema (HTTP)	Integración	Levanta el servidor y ejecuta CP-07 y CP-08 por HTTP.
Aceptación	Sistema (HTTP)	Ejecuta los escenarios de la cliente CP-09 y CP-10.
Regresión con DEF-01 y DEF-02 inyectados	Aceptación	Inyecta DEF-01 y DEF-02 y falla si las pruebas ya no los detectan.
Reporte de resultados	Todos (se ejecuta siempre)	Junta los JUnit, publica la tabla de resultados en el resumen del job y sube el artefacto «evidencia-pruebas».

Reglas del repositorio que completan el flujo:

- Protección de ramas en main y develop (activa desde el 30 de octubre de 2026): no se permite push directo, ni siquiera al administrador; todo entra por Pull Request y GitHub no permite el merge si falla alguno de los 7 checks (Unitarias en PHP 8.2, 8.3 y 8.4, Integración, Sistema (HTTP), Aceptación y Regresión). GitHub no exige una aprobación porque el repositorio está en una sola cuenta; la revisión del PR le corresponde a Emily.
- concurrency: si llega un push nuevo a la misma rama, la ejecución anterior se cancela.
- Caché de Composer por versión de PHP y acción compuesta .github/actions/preparar-php para no repetir la instalación en cada job.
- La evidencia de cada ejecución queda descargable como artefacto, lo que sirve para adjuntarla al defecto en Trello.

Fragmento del workflow:

```
on:
  push:
    branches: [main, develop, "feature/**", "release/**", "hotfix/**"]
```

```
  pull_request:
    branches: [main, develop]
  schedule:
    - cron: "0 7 * * *" # 01:00 en Campeche (UTC-6)
  workflow_dispatch:

concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true

permissions:
  contents: read

jobs:
  unitarias:
    name: Unitarias (PHP ${{ matrix.php }})
    runs-on: ubuntu-latest
    timeout-minutes: 10
    strategy:
      fail-fast: true
      matrix:
        php: ["8.2", "8.3", "8.4"]
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/preparar-php
        with:
          php-version: ${{ matrix.php }}

      - name: CP-01 a CP-04
        run: php scripts/ejecutar-pruebas.php --suite=Unitarias

      - uses: actions/upload-artifact@v4
        if: always() && matrix.php == '8.2'
        with:
          name: junit-Unitarias
          path: evidencias/junit-Unitarias.xml
```

Anexo. Cómo ejecutar

Comando	Para qué
composer test	Ejecuta los 10 casos en la consola.
composer test:unitarias	Ejecuta un solo nivel (o test:integracion, test:sistema y test:aceptacion).
composer evidencia	Ejecuta las cuatro suites y genera la evidencia en evidencias/.
composer regresion	Inyecta DEF-01 y DEF-02 y comprueba que las pruebas los detectan.
php -S localhost:8000 -t public	Levanta la API del sistema (/api/productos, /api/reporte, /salud).